Name:	Krish Sodhi, Marvin Sukumar, Riya Suryawanshi, Anushka Suvarna
UID:	2023300233, 2023300238, 2023300240, 2023300241
Subject:	Distributed Computing
Experiment No.:	3
Project title:	Travel Itinerary Planner
Problem Statement:	Implementation of Multithreading in Distributed System
Objectives:	To implement a multithreaded server that handles multiple client requests simultaneously and applies multithreading to fetch different itinerary components (restaurants, events, attractions, and transport) in parallel for faster and more responsive travel planning.
Theory:	Plagiarism is a serious issue in academic institutions and affects the quality and credibility of education. It occurs when a student copies someone else's work and presents it as their own without proper acknowledgment. With the increase in digital learning and online submissions, plagiarism has become easier to commit and harder to detect manually. Therefore, automated plagiarism detection systems are very important for schools, colleges, and universities. A plagiarism detection system works by comparing one document with many other documents to find similarities. Instead of reading each submission manually, the system uses computer algorithms to analyze the text. This saves time for teachers and ensures fair evaluation of student work. These systems can detect copied sentences, paragraphs, and sometimes even paraphrased content. One commonly used technique for plagiarism detection is TF-IDF combined with cosine similarity. TF-IDF stands for Term Frequency–Inverse Document Frequency. It helps the computer understand which words in a document are important. Common words like "the", "is", and "and" appear in almost every document, so they are given very low importance. On the other hand, subject-specific words such as "plagiarism", "algorithm", and "detection" are given higher importance because they carry more meaning.

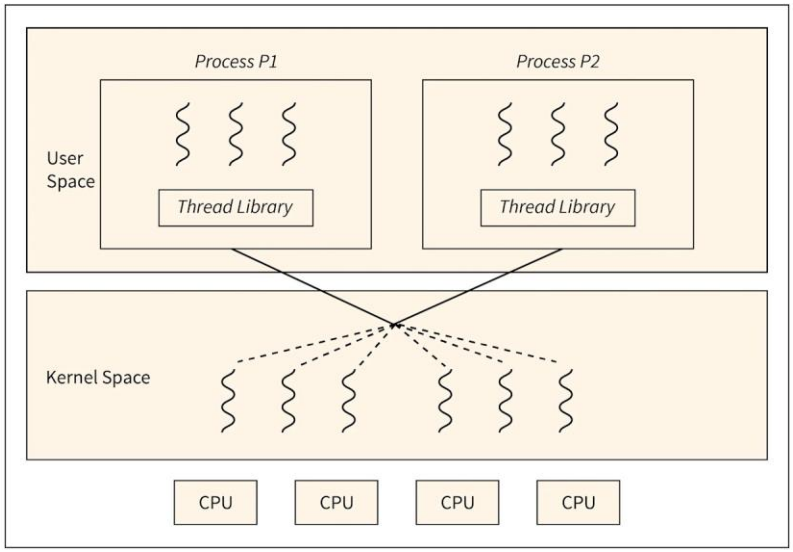


Figure 1: Multithreading architecture showing user-level threads managed by the thread library and mapped to kernel threads for execution on multiple CPUs.

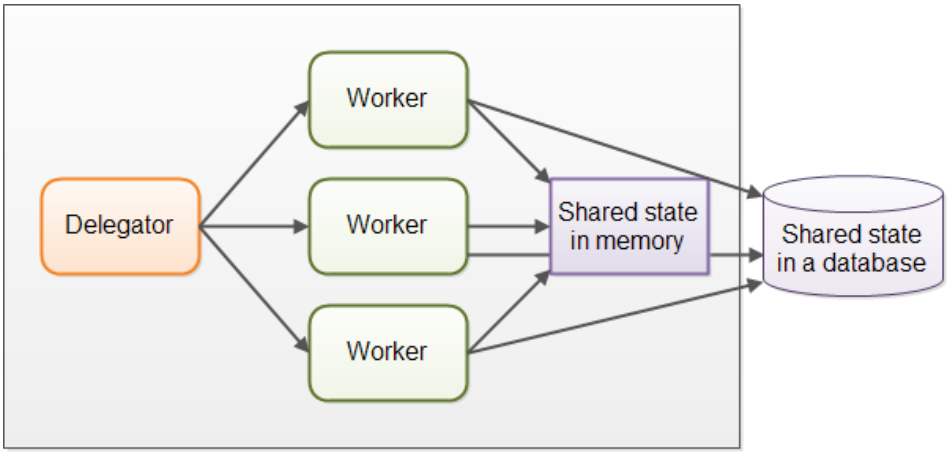


Figure 2: A Fine-Grain Multithreading Superscalar Architecture by Loikkanen and Bagherzadeh:

	In our Travel Itinerary Planner server, multithreading enables the system to handle multiple client connections and multiple services simultaneously. Each client request runs in its own thread. This ensures that many users can interact with the system at the same time without waiting for others, improving responsiveness and scalability. For example: ● One thread can fetch restaurants from the database. ● Another thread can retrieve events happening at the same location. ● A separate thread can list attractions nearby. ● Another thread can check transport options between locations. Because these threads run in parallel, itinerary generation becomes much faster compared to sequential execution. Even if fetching transport data takes longer, other services (like restaurants or attractions) are not blocked. Multithreading also improves the user experience: ● While one user logs in, another can search for events. ● A third user can plan transport options without affecting the others. ● Background threads keep updating data (such as live transport or event availability) while the main thread handles user interactions. Thus, with multithreading, my system supports real-time, distributed itinerary planning where restaurants, events, attractions, and transport information are all processed simultaneously and combined into a complete travel plan.

Code:	**Server:-**
	`import threading`
	`import socket`
	`import time`
	`import json`
	`import sqlite3`
	`from datetime import datetime`
	
	`HOST = "127.0.0.1"`
	`PORT = 5000`
	
	`def client_request(conn, addr):`
	`print(f"[NEW CONNECTION] {addr}")`
	`print(f"Active threads: {threading.active_count() -`

```
1}")
    data = conn.recv(1024).decode()
    msg = json.loads(data)
    if msg.get("req") == "restaurants":
        location = msg.get("location", "Goa")
        min_price = msg.get("min", 0)
        max_price = msg.get("max", 100000)
        db_conn = sqlite3.connect("Main_DB.db")
        curr = db_conn.cursor()
        results = curr.execute(
            "SELECT name, price FROM restaurants WHERE
location=? AND price BETWEEN ? AND ?",
            (location, min_price, max_price)
        ).fetchall()
        db_conn.close()
        time.sleep(80)
        if results:
            response = [[name, price] for name, price in
results]
            conn.send(json.dumps(response).encode())
        else:
            conn.send("No results".encode())

    elif msg.get("req") == "transport":
        curr_loc = msg.get("Current")
        dest = msg.get("Destination")
        db_conn = sqlite3.connect("Main_DB.db")
        cur = db_conn.cursor()
        results = cur.execute(
            "SELECT
start_location,end_location,airline,price FROM
transport_routes WHERE start_location=? AND end_location=?
ORDER BY price ASC",
            (curr_loc, dest)
        ).fetchall()
        db_conn.close()
        time.sleep(80)
        if results:
            response = [[start_location, end_location,
```

```
airline, price] for start_location, end_location, airline,
price in results]
            conn.send(json.dumps(response).encode())
        else:
            conn.send("No results".encode())

    elif msg.get("req") == "attractions":
        loc = msg.get("location")
        type_ = msg.get("type", "")
        db_conn = sqlite3.connect("Main_DB.db")
        cur = db_conn.cursor()
        if not type_:
            results = cur.execute(
                "SELECT name,location,category FROM
attractions WHERE location=?",
                (loc,)
            ).fetchall()
        else:
            results = cur.execute(
                "SELECT name,location,category FROM
attractions WHERE location=? AND type=?",
                (loc, type_)
            ).fetchall()
        db_conn.close()
        time.sleep(80)
        if results:
            response = [[name, location, category] for
name, location, category in results]
            conn.send(json.dumps(response).encode())
        else:
            conn.send("No results".encode())

    elif msg.get("req") == "events":
        start = msg.get("start")
        loc= msg.get("location")
        end = msg.get("end")
        start_date = datetime.strptime(start,
"%d-%m-%Y").strftime("%Y-%m-%d")
        end_date = datetime.strptime(end,
```

```
"%d-%m-%Y").strftime("%Y-%m-%d")
        db_conn = sqlite3.connect("Main_DB.db")
        cur = db_conn.cursor()
        results = cur.execute(
            "SELECT name,location,date,category FROM
events WHERE date >= ? AND date <=? AND location=?",
            (start_date, end_date,loc)
        ).fetchall()
        db_conn.close()
        time.sleep(80)
        if results:
            response = [[name, location, date, category]
for name, location, date, category in results]
            conn.send(json.dumps(response).encode())
        else:
            conn.send("No results".encode())

    conn.close()
    print(f"[DISCONNECTED] {addr}")

def start_server():
    server = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen()
    print(f"[LISTENING] on {HOST}:{PORT}")
    while True:
        conn, addr = server.accept()
        thread = threading.Thread(target=client_request,
args=(conn, addr))
        thread.start()

start_server()
```

Client:-

```
import socket
import json
```

```
def rest(loc,min_price,max_price):

client=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
    client.connect(("localhost",5000))
    data = {
        "req":"restaurants",
        "location":loc,
        "min":min_price,
        "max":max_price
    }
    client.send(json.dumps(data).encode())
    data = client.recv(1024).decode()
    print(data)
    client.close()

def transport(curr_loc,dest):

client=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
    client.connect(("localhost",5000))
    data = {
        "req":"transport",
        "Current":curr_loc,
        "Destination":dest
    }
    client.send(json.dumps(data).encode())
    data = client.recv(1024).decode()
    print(data)
    client.close()

def attractions(location,type:str=None):

client=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
    client.connect(("localhost",5000))
    data = {
        "req":"attractions",
        "location":location,
```

```
        "type":type if type else ""
    }
    client.send(json.dumps(data).encode())
    data = client.recv(1024).decode()
    print(data)
    client.close()

def events(loc, start,end):

client=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
    client.connect(("localhost",5000))
    data = {
        "req":"events",
        "location":loc,
        "start":start,
        "end":end
    }
    client.send(json.dumps(data).encode())
    data = client.recv(1024).decode()
    print(data)
    client.close()
ch=1
while(ch!=3):

    print('''

       Menu
       1. Restaurants/Hotels
       2. Events
       3. Transport Routes
       4. Attractions

    '''
    )

    ch=int(input("Enter your choice: "))

    if(ch==1):
        loc=str(input("Enter desired location: "))
```

```
        max = int(input("Enter Max Price:"))
        rest(loc,0,max)

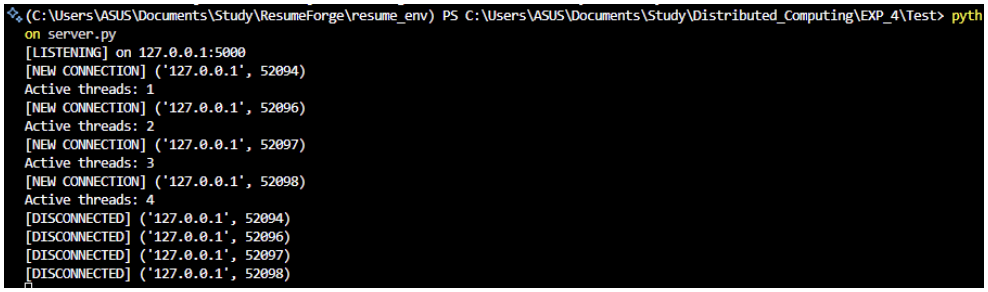

    elif(ch==2):

        loc=str(input("Enter desired location: "))
        start = str(input("Enter journey start date
(e.g.20-09-2025):"))
        end =str(input("Enter journey end date
(e.g.20-09-2025):"))
        events(loc,start,end)

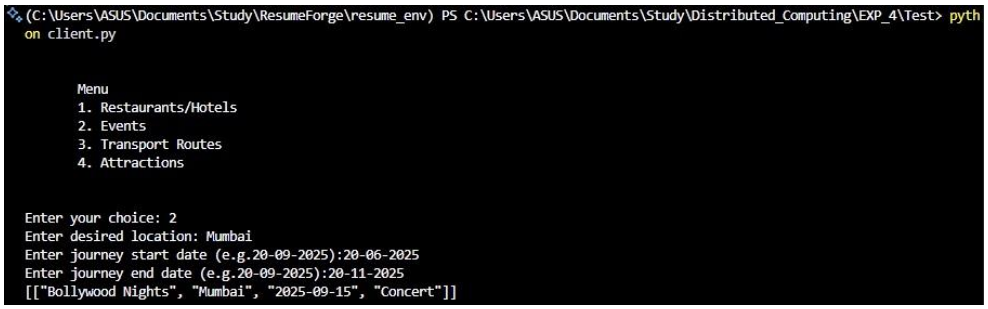
    elif(ch==3):
        curr = str(input("Enter journey start location:"))
        dest = str(input("Enter journey end
destination:"))
        transport(curr,dest)

    elif(ch==4):
        loc=str(input("Enter desired location: "))
        type_att = str(input("Enter type of
attraction:(optional, Press enter if nothing)"))
        attractions(loc,type_att)
    else:
        print("Thank You! Bye")
```

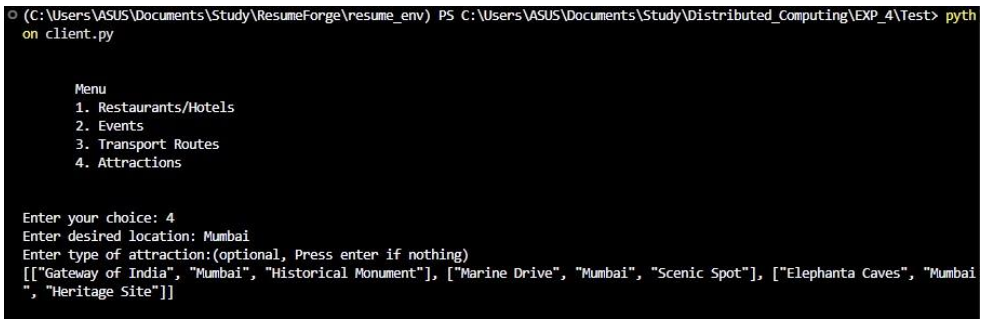
Output:



<table>
<tr>
<td></td>
<td>





</td>
</tr>
<tr>
<td>Conclusion:</td>
<td>This experiment showed how multithreading makes a distributed system faster and more responsive by handling many client requests at the same time. In the travel itinerary planner, it helped fetch restaurants, events, attractions, and transport details in parallel, giving users a smooth and efficient experience.</td>
</tr>
</table>